

Architecture d'héritage des templates Django

BioAttend - Structure base.html imbriquée

Table des matières

1. Vue d'ensemble de la structure
2. Les trois fichiers base.html
3. Flux d'héritage
4. Comment implémenter les pages du dashboard
5. Exemples pratiques

Vue d'ensemble de la structure

Votre projet utilise une structure d'héritage de templates imbriquée, souvent appelée **"poupée russe"** car chaque template enveloppe les éléments des templates qui l'héritent. Cela crée une hiérarchie claire et maintenable.

Les avantages principaux :

- Évite la duplication du code HTML
- Permet de partager les assets (CSS, JS) facilement
- Maintenabilité : une modification au niveau base affecte tous les enfants
- Séparation claire entre pages d'authentification et dashboard

Les trois fichiers base.html

Votre projet contient trois fichiers base.html distincts, chacun avec un rôle spécifique :

Fichier	Localisation	Rôle
core/base.html	core/templates/	Base racine du site. HTML structurel minimal.
dashboard/base.html	dashboard/templates/	Enveloppe core/base.html. Ajoute layout + sidebar.
dashboard/dashboard/base.html	dashboard/templates/dashboard/	Duplique la structure. À NETTOYER.

1. core/templates/base.html

C'est la fondation minimale. Contient :

- Doctype, balises meta standard
- CSS global (auth.css)
- Blocs extensibles : extra_css, extra_js, content

Utilisée par les pages d'authentification et d'autres pages simples.

2. dashboard/templates/base.html

```
{% extends "core/base.html" %}
```

C'est le template intermédiaire qui ajoute la structure du dashboard. Il :

- Hérite de core/base.html
- Ajoute Font Awesome, style.css et app.js
- Fournit la structure HTML : app-container, sidebar, main-content
- Définit un bloc dashboard_content pour le contenu de chaque page

Toutes les pages du dashboard héritent de ce template.

3. dashboard/templates/dashboard/base.html

ATTENTION

Ce fichier duplique la structure de dashboard/templates/base.html. Pour une meilleure maintenabilité, supprimez ce fichier et mettez à jour les pages qui l'utilisaient.

Flux d'héritage

Voici comment les templates s'imbriquent :

core/templates/base.html

↓

dashboard/templates/base.html

{% extends "core/base.html" %}}, ajoute structure du dashboard

↓

Pages du dashboard (ex: users.html, reports.html)

{% extends "dashboard/base.html" %}}, remplissent dashboard_content

Comment implémenter les pages du dashboard

Chaque page du dashboard doit suivre ce modèle :

dashboard/templates/pages/users.html

```
{% extends "dashboard/base.html" %}
{% load static %}

{% block title %}Users - BioAttend{% endblock %}

{% block extra_css %}
    <link rel="stylesheet" href="{% static 'css/users.css' %}">
{% endblock %}

{% block dashboard_content %}
    <h1>Gestion des utilisateurs</h1>
    <!-- Contenu spécifique de la page -->
    <table>...</table>
{% endblock %}

{% block extra_js %}
```

```
<script src="{% static 'js/users.js' %}"></script>
{% endblock %}
```

Points clés

- **{% extends "dashboard/base.html" %}** - Hérite de ce template
- **{% block title %}** - Change le titre de la page (optionnel)
- **{% block extra_css %}** - Ajoute des CSS spécifiques à cette page
- **{% block dashboard_content %}** - **BLOC PRINCIPAL** pour le contenu unique
- **{% block extra_js %}** - Ajoute des JS spécifiques à cette page

Exemples pratiques

Exemple 1 : Page de liste d'utilisateurs

```
{% extends "dashboard/base.html" %}
{% load static %}

{% block title %}Utilisateurs{% endblock %}

{% block dashboard_content %}
    <div class="page-header">
        <h1>Gestion des utilisateurs</h1>
        <button class="btn btn-primary">Ajouter un utilisateur</button>
    </div>

    <table class="users-table">
        <thead>
            <tr>
                <th>Nom</th>
                <th>Email</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            {% for user in users %}
            <tr>
                <td>{{ user.get_full_name }}</td>
                <td>{{ user.email }}</td>
                <td>
                    <a href="#">Éditer</a> | <a href="#">Supprimer</a>
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
{% endblock %}
```

Exemple 2 : Page de rapports avec graphiques

```

{% extends "dashboard/base.html" %}
{% load static %}

{% block title %}Rapports{% endblock %}

{% block extra_css %}
    <link rel="stylesheet" href="{% static 'css/charts.css' %}">
{% endblock %}

{% block dashboard_content %}
    <h1>Rapports d'assiduité</h1>

    <div class="charts-container">
        <canvas id="attendanceChart"></canvas>
        <canvas id="presenceChart"></canvas>
    </div>
{% endblock %}

{% block extra_js %}
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <script src="{% static 'js/charts.js' %}"></script>
{% endblock %}

```

Bonnes pratiques

✓ À faire

- Toujours commencer par `{% extends "dashboard/base.html" %}`
- Utiliser `{% block dashboard_content %}` pour le contenu principal
- Ajouter des CSS/JS spécifiques dans `extra_css` et `extra_js`
- Nommer les fichiers clairement : `users.html`, `reports.html`, `settings.html`
- Mettre en place une structure cohérente : `dashboard/templates/pages/`

✗ À éviter

- Ne pas dupliquer la structure du dashboard (supprimer `dashboard/templates/dashboard/base.html`)
- Ne pas importer directement `core/base.html` dans les pages du dashboard
- Ne pas ajouter de CSS/JS global dans les `extra_css/extra_js`
- Ne pas modifier `dashboard/base.html` pour une page spécifique

Structure de dossiers recommandée

```

dashboard/
  templates/
    base.html (hérite de core/base.html)
  pages/
    users.html
    reports.html
    settings.html
  partials/

```

```
_sidebar.html  
_topbar.html  
components/ (boutons, modales, etc.)
```

Conclusion

Votre architecture d'héritage de templates est bien structurée. En suivant ces recommandations :

6. Toutes les pages du dashboard auront une apparence et une structure cohérentes
 7. Les modifications au template de base affecteront instantanément toutes les pages
 8. Chaque page peut avoir ses propres CSS et JS sans polluer les autres
 9. Le code reste facile à maintenir et à évoluer
-